

Architecture

I built a small system that learns a browser workflow once and reuses it without asking a model to plan each run. The example is a savings-balance lookup in a Flask app with two fictional members. Python handles orchestration, and Playwright drives Chromium. One asyncio process owns the browser so actions, timeouts, and control transfers have a clear order.

Discovery and replay share an executor. Before an action reaches the browser, the executor validates its contract, binds inputs, checks policy and ownership, and applies a deadline. After execution it checks conditions and records a sanitized result. Keeping those checks in one place prevents the two modes from developing different permission rules.

Discovery uses Gemini or OpenAI to select actions from a reviewed control vocabulary and live visibility/state observations. Inputs are represented as references rather than member values in the model context. The model chooses the sequence; it does not invent locators on an unknown site. This constraint made it possible to test the complete workflow and its failure cases.

Artifact schema

A capability is a JSON document with a schema version, capability ID and version, compatible target, typed inputs and outputs, ordered steps, and a success checkpoint. It can also declare business outcomes and bounded wait recovery. Each step includes an operation, action, target, and timeout. The loader rejects duplicate keys, oversized files, invalid references, and unsupported versions.

I kept member IDs and decimal balances as strings to preserve leading zeros and avoid floating-point rounding. Compilation records only actions that executed successfully. Input references come from matching reviewed templates, not global replacement of strings that happen to match an example value. The resulting artifact is independent of the model transcript.

Generated artifacts carry a discovery run ID. Hand-authored fixtures are labelled separately. Run IDs and file hashes let a reviewer correlate the evidence, but they are not a provider-signed attestation.

Determinism & error handling

Replay binds new inputs and follows the stored sequence without model calls. Targets must match exactly one control. Scoped roles and labels are preferred; reviewed CSS handles the balance cells. Missing or ambiguous matches stop the run. Success requires typed outputs and a visible checkpoint identifying the requested account owner.

Results distinguish success, a business outcome such as member-not-found, and failure. Permission denial, expired sessions, application errors, invalid inputs, and failed checkpoints retain separate codes. Recovery is limited to bounded condition waits for known loading states. It never resubmits a search or repeats a click to see whether it works.

Discovery also has step, elapsed-time, invalid-response, and no-progress limits. A model cannot declare an answer without UI evidence. Errors retain the stopped step and expected/observed context. Failure to store diagnostics closes the browser.

Heterogeneity & multi-tenant

The controllers use a surface interface rather than Playwright objects. A desktop adapter could implement the same operations with OS accessibility APIs. Legacy frames or inaccessible visual controls would need explicit scopes or reviewed visual targeting, plus their own boundary tests. Those adapters are not built.

Tenant reuse would keep the workflow while applying reviewed origin, target identity, locator, and operation settings for a compatible application version. Each run already has a separate browser context.

Product/version checks reject incompatible pages, although the current markers assume a cooperating app and are not authentication.

I did not build a tenant registry, credential vault, or compatibility matrix. Those would be needed before claiming safe reuse across production tenants.

Escalation & handoff

When an interactive run is blocked, a local operator page shows the goal, stopped action, reason, and sanitized context. The existing Chromium session stays open. Control moves from automation to awaiting-human, human-control, and resume-check. Automation dispatch is blocked while the person owns the session.

Browser listeners record interaction categories and element types, not field contents or keystrokes. Before resuming, the coordinator checks target identity, the exact member's accounts route, owner checkpoint, and output controls. It then grants a single-use resume permission and rereads the outputs. A wrong page or an unresolved dialog remains paused. Cancel and operator deadlines end the run.

This resume strategy is specific to the savings workflow. Human-assisted discovery can return verified outputs, but it does not compile a partial model history into a capability that pretends to include the person's actions.

Safety

Both modes use a default-deny allowlist for origins, routes, actions, and operations. The executor checks actual controls and form destinations. Chromium request interception blocks unapproved redirects, frames, popups, background APIs, WebSockets, and downloads. A separate human rule permits only the current member's CSRF-checked dialog acknowledgement.

Logs replace unreviewed text and registered inputs/outputs with per-run keyed tokens. Failure snapshots keep DOM hierarchy, tags, visibility, and disabled state, without text, values, URLs, cookies, or scripts. Runtime capture saves no raw screenshots or traces. Manually collected demo images are reviewed before publication and labelled as synthetic data.

The operator token restricts local commands; it does not verify a person's identity. Browser ownership checks cannot lock the OS, address bar, developer tools, or extensions. These are boundaries for a local demonstration, not a production banking security model.

Cuts

The completed example covers discovery, generated-artifact reuse, model-free replay, error handling, policy, redaction, and same-session handoff. The latest manual demonstration includes six model responses, replay for the second member, and a separate handoff using the earlier capability. Fresh-environment acceptance also records exceptional CLI outcomes and retains the first failed attempt.

Historical runs did not capture a full source snapshot at discovery time. Acceptance archives identify the source they tested; they cannot establish an older run's exact revision. The checks reuse the same macOS host and can reuse download caches.

I kept the scope to one read-only workflow. Arbitrary-site exploration, desktop execution, account-changing operations, multi-user control, and general workflow repair remain outside this version. My next extension would be a second target adapter, with the same contracts and acceptance checks.